

Agentic RAG for Formula 1

A retrieval-augmented assistant that reasons, plans and self-corrects — built by hand with LangGraph.

LangGraph

Hybrid retrieval

Bilingual FR / EN

Groq · ONNX embeddings

1 What I built, and why

I wanted an assistant that does more than paste back the first paragraph it finds. The goal of this project was a full **Agentic RAG** system: one that decides which tools to use, checks whether the documents it retrieved actually answer the question, rewrites its own query when they don't, and verifies that its final answer is backed by the sources before returning it.

The whole reasoning loop is **written by hand with LangGraph**, not with the ready-made `create_agent` helper — that was the point of the exercise, and it let me insert the extra steps (planning, grading, verification) that a pre-built ReAct loop hides. I chose **Formula 1** as the domain because it mixes short factual questions (“what is DRS?”) with genuinely hard ones (comparing two drivers, tracing a rule across seasons, doing arithmetic on championship points) — exactly the contrast needed to show the difference between plain RAG and an agentic one. The system is **bilingual**: you can ask in French or English over a mostly-English corpus.

2 How the system is put together

Knowledge base & vectorization

83 Wikipedia pages (≈ 3.0 M characters) are downloaded, cleaned and sorted into **9 categories** — regulations, teams, drivers, circuits, glossary, technology, **strategy**, seasons (every year 2005–2025) and history. They are split into **6,493 chunks**, each prefixed with its title and section so it stays self-explanatory. Embeddings are computed **locally** with a multilingual model running on **ONNX / FastEmbed**, then stored in **Chroma**. No embedding API, no key.

Tools the agent can call

- `rechercher_documents` — hybrid search over the whole corpus;
- `rechercher_par_categorie` — search filtered to one category;
- `comparer_entites` — pulls facts on two entities in parallel;
- `inventaire_corpus` — lists what the base contains;
- `calculer_points_championnat` — *deterministic* points math.

The LLM is interchangeable through a small factory (Groq, Gemini or local Ollama) — you just edit `.env`. The heavy reasoning uses Llama 3.3 70B; the high-volume grading roles use a lighter 8B model.

3 The reasoning graph

Every node is a small pure function that reads the shared *state* and returns only what it changes; reducers handle merging (including de-duplicating retrieved chunks). The state also carries the conversation history, which is what gives the agent its **memory** — a follow-up like “and his team-mate?” is rewritten into a standalone question before anything else happens.

Beyond a linear RAG, the graph adds four agentic behaviours: it *analyses* and routes the question, *plans* complex ones into sub-questions, *grades* the retrieved documents and *rewrites* the query if they miss (CRAG-style self-correction), and finally *verifies* that the answer is grounded — regenerating it if not. Budgets cap every loop so it always terminates.

```
START
↓
analyze_query (language, rewrite,
              simple / complex)
↓
plan_research (if complex)
↓
agent -----> generate
  ↑ (relevant)   ↓
tools → grade_docs verify_grounding
      ↓ (off-topic) ↓
      rewrite_query  END
```

4 Problems I hit — and how I solved them

The interesting part of the project wasn't the happy path, it was everything that broke:

Problem	What I did about it
PyTorch wouldn't install	No wheel exists for macOS x86_64 on Python 3.13, so <code>sentence-transformers</code> was a dead end. I switched embeddings to ONNX / FastEmbed — no PyTorch, lighter, and faster on CPU.
Generic text ranked above the real answer	FastEmbed skipped the final vector normalization, so similarity was dominated by vector length and long generic passages always won. I normalize the vectors myself , restoring a true cosine.
Acronyms & names got lost	Dense search misses literal tokens like DRS or ERS. I added BM25 alongside it and fuse the two with Reciprocal Rank Fusion , which combines ranks without normalizing scores.
One page swamped every result	The “Formula 1” page alone is ~390 chunks. I cap chunks per document so a generalist page can't monopolize the top-k.
“2010 champion” returned the wrong year	With 22 near-identical season pages, the year token got diluted. A deterministic year boost re-anchors the search on the requested season — verified on every year I tested.
The LLM sometimes crashed the graph	Llama via Groq occasionally emits a malformed tool call (a 400). Every LLM node now catches errors and degrades gracefully (fail-open) instead of taking down the whole run.
Free-tier token & rate limits	I route the high-volume grading/judge roles to a cheaper 8B model and made the evaluation save incrementally and merge , so a network drop mid-run doesn't lose completed questions.

5 Results

The system is evaluated on **10 simple and 10 complex** bilingual questions, measuring three things: end-to-end **latency**, **document relevance** (`hit@k`, `MRR`), and **answer quality** scored out of 5 by an LLM judge (faithfulness to sources, completeness, clarity).

0.68

HIT@K

0.56

MRR

3.5

FAITHFULNESS /5

4.0

CLARITY /5

111

LATENCY (S)

Category	Latency (s)	hit@k	MRR	Faithfulness /5	Completeness /5	Clarity /5
Simple questions (10)	111.73	0.65	0.46	3.00	2.90	3.50
Complex questions (10)	110.62	0.70	0.65	4.10	3.40	4.50
Overall (20)	111.18	0.68	0.56	3.55	3.15	4.00

Model: `groq:llama-3.1-8b-instant` (roles legers: `llama-3.1-8b-instant`) · 20 questions evaluated. The evaluation above ran on the light model (Llama 3.1 8B) to stay within the free daily quota; the target setup uses Llama 3.3 70B for reasoning.

A concrete run — “How many world titles has Max Verstappen won?”

analyze_query → agent → tools (search) → grade_docs (relevant) → agent → generate → verify_grounding (grounded) → END

Answer. “Max Verstappen has won four Formula 1 world titles [SOURCE 6], taken consecutively from 2021 to 2024 with Red Bull [SOURCE 6].” — grounded in the sources, with citations, in the language of the question.

Reading the table. `hit@k` is the share of expected documents actually retrieved; `MRR` rewards ranking a relevant source first. Latencies are inflated by free-tier rate-limit back-off and by the agentic loop's multiple LLM calls, not by the retrieval itself.

6 Limits & where I'd take it next

Limits

- The embedding model is **symmetric**; dense search still occasionally prefers generic passages, softened but not erased by BM25 and category filtering.
- Malformed LLM tool calls are handled, but a caught call means a slightly less complete answer.
- **Latency**: the agentic loop multiplies LLM calls (~30 s per complex question on 70B).
- The corpus is **static** — no live race results.

Next steps

- Move to an **asymmetric** embedding model (e5, BGE) for better question→passage recall.
- Add a **cross-encoder re-ranker** after fusion to sharpen the top-k.
- **Stream** the final answer token by token to cut perceived latency.
- Persist memory to **disk (SQLite)** for durable sessions.
- Plug in **live data sources** for current-events questions.